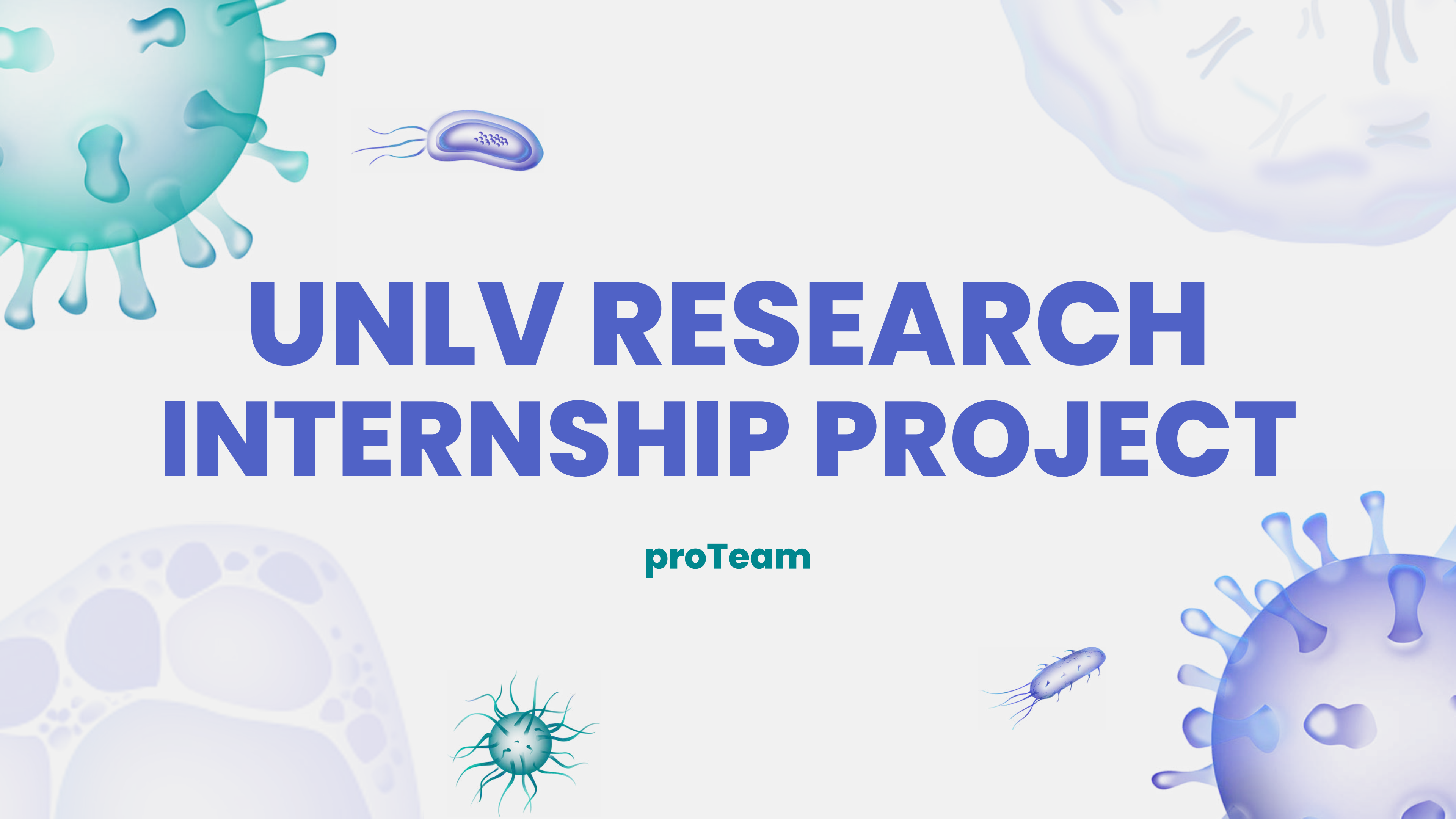




UNLV RESEARCH INTERNSHIP PROJECT

proTeam

Table of Contents

01 Introduction

02 Topic Significance

03 Benchmark Models

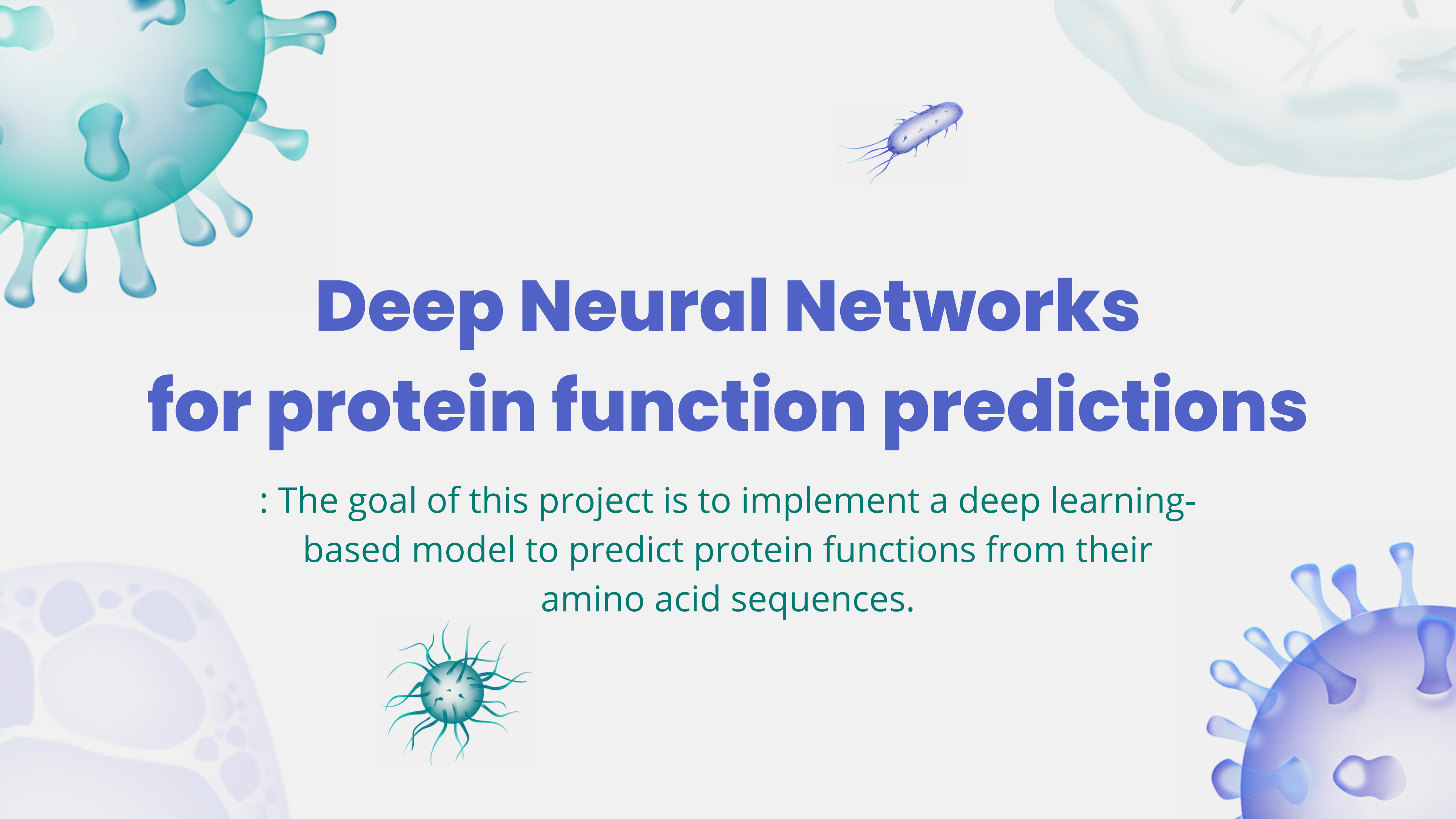
04 Experimental Design

05 Experimental Results

06 Model Interpretation

07 Discussion

08 Reference



Deep Neural Networks for protein function predictions

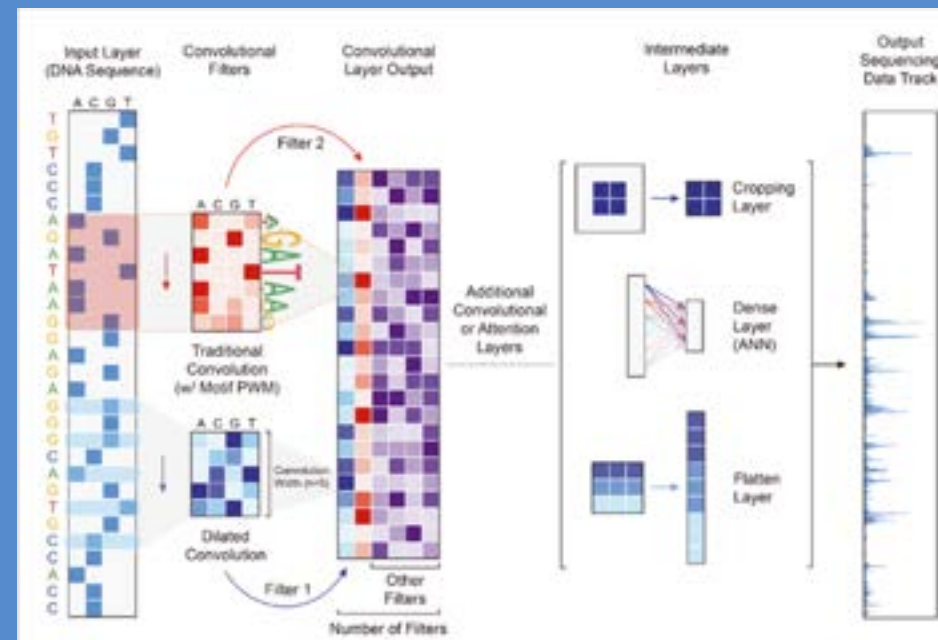
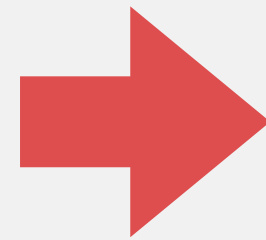
: The goal of this project is to implement a deep learning-based model to predict protein functions from their amino acid sequences.

Introduction

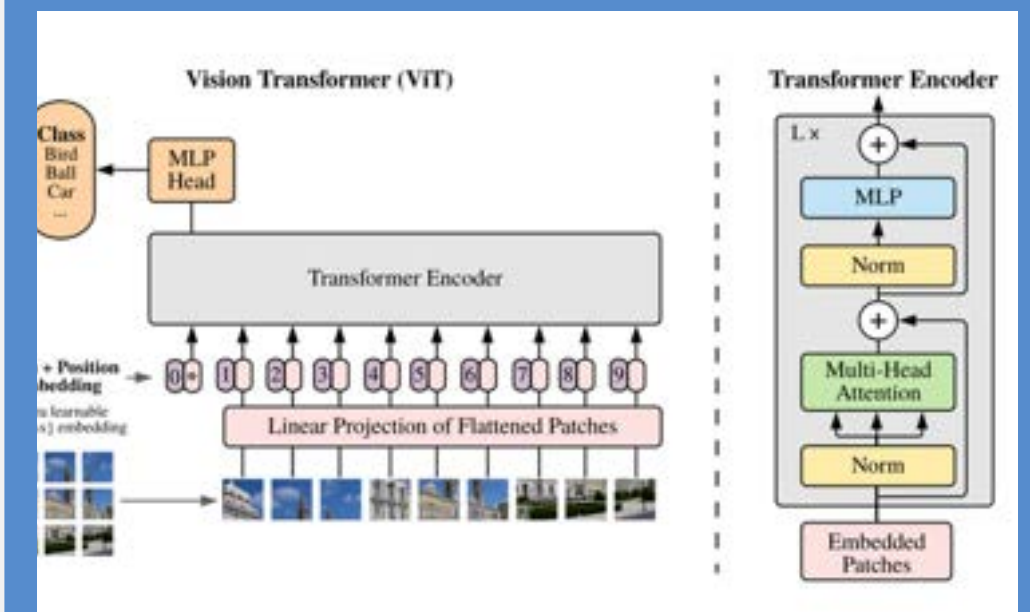
One-Hot

K-mer

Tokenization



CNN Model

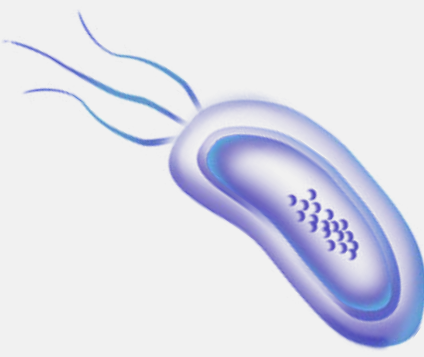


Transformer Model

Comparing CNN and Transformer models for EC number prediction using different protein sequence encoding methods : one-hot, k-mer, and tokenization.

Topic Significance

EC Numbers: A Functional Hierarchy



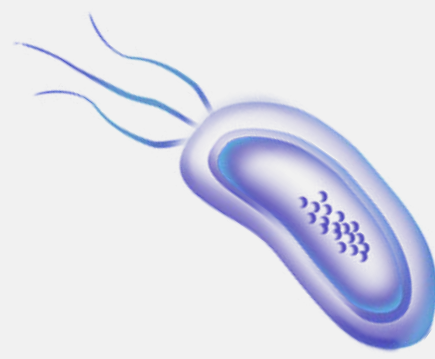
EC (Enzyme Commission) numbers classify enzymes into four functional levels.

Format: x.y.z.w

> More digits = more specific reaction.

They're not just names, but they reflect functional similarity between enzymes.





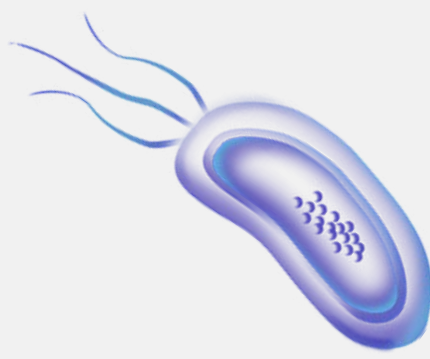
4-Level Structure of EC Numbers (x.y.z.w)

Level	Meaning
x (1st)	General class of reaction (e.g., oxidoreductase, transferase)
y (2nd)	More specific group of reactions
z (3rd)	Even more detailed reaction type
w (4th)	Specific substrate or mechanism

More matching digits → **Higher** functional similarity

EC number similarity is a quantitative clue to enzyme function





ProTeam's Strategy

If higher levels (like x or x.y) match, it's still considered partially correct.

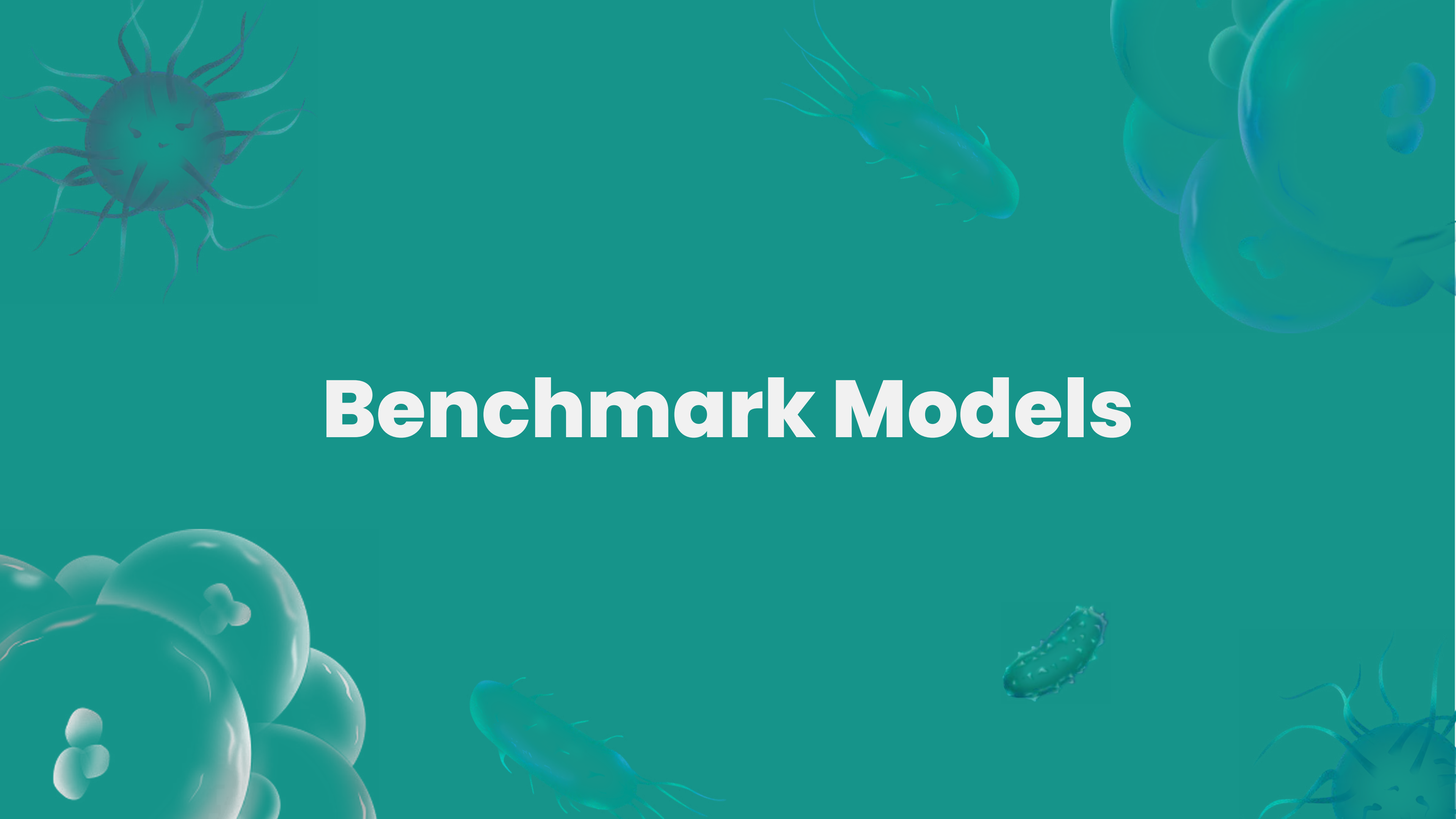
Why?

- Binary evaluation = even similar predictions are marked wrong
- Hierarchy-aware evaluation = more biologically meaningful

Our team fixed the **first digit as '1'**

And checked similarity up to the **second digit**.



The background is a solid teal color with several 3D-rendered microscopic organisms. In the top left, there is a large, spherical, pinkish-purple cell with many thin, radiating filaments. In the top right, there are several large, translucent, light blue spherical cells, some of which contain smaller, darker blue structures. In the bottom left, there is a cluster of similar light blue spherical cells. In the bottom right, there is a large, spherical, pinkish-purple cell with radiating filaments, similar to the one in the top left. Scattered throughout the background are several rod-shaped bacteria, some in blue and some in pink, with short, hair-like appendages (flagella) extending from their surfaces.

Benchmark Models

Benchmark Models

JOURNAL ARTICLE

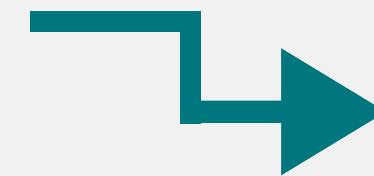
DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome FREE

[Yanrong Ji](#), [Zhihan Zhou](#), [Han Liu](#) ✉, [Ramana V Davuluri](#) ✉ [Author Notes](#)

Bioinformatics, Volume 37, Issue 15, August 2021, Pages 2112–2120,



K-mer



Tokenization

ECPIK

Powered by [DataX Lab](#) Powered by [CPS Lab](#)

python [3.6](#) | [3.7](#) | [3.8](#) | [3.9](#) | pypi [v0.0.1](#) | pypi downloads [18/month](#) | license [MIT](#)

Biologically interpretable deep learning enhances trustworthy enzyme commission number prediction and discovers potential motif sites



One-Hot

DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome (2021)

datax-lab. (2023). ECPIK: Biologically interpretable deep learning enhances trustworthy enzyme commission number prediction and discovers potential motif sites [Source code]. GitHub.

Benchmark Models

[Submitted on 22 Oct 2020 (v1), last revised 3 Jun 2021 (this version, v2)]

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision are either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping the overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can achieve state-of-the-art results on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (e.g., ImageNet, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially less resources to train.

Comments: Fine-tuning code and pre-trained models are available at [this https URL](https://github.com/google-research/vision_transformer). ICLR camera-ready version with 2 small modifications: 1) Added a discussion of



Transformer

ECPIK

Powered by DataX Lab Powered by CPS Lab

python 3.6 | 3.7 | 3.8 | 3.9 pypi v0.0.1 pypi downloads 18/month license MIT

Biologically interpretable deep learning enhances trustworthy enzyme commission number prediction and discovers potential motif sites



CNN

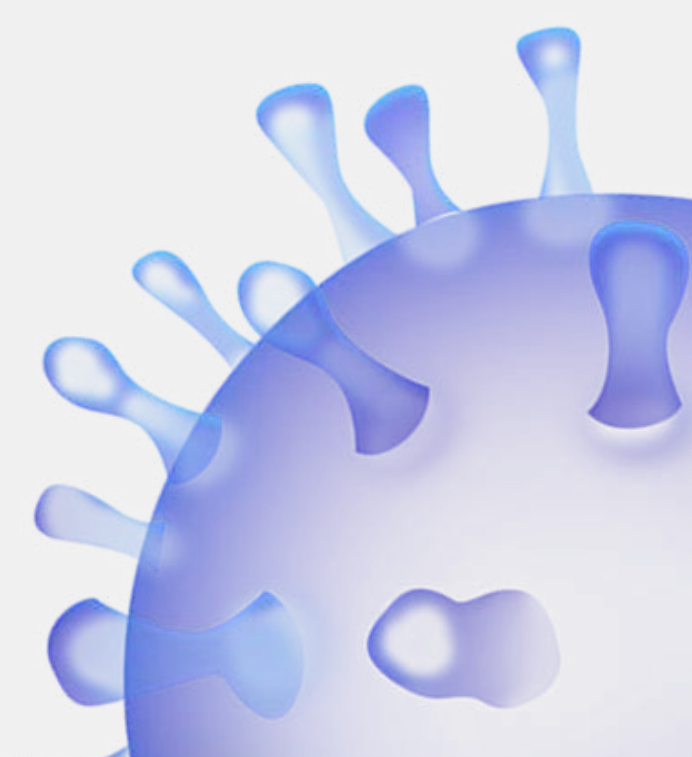
The background is a solid blue color. It features several stylized, semi-transparent illustrations of microscopic organisms. In the top left, there is a spherical organism with many thin, radiating lines. In the top right, there is a cluster of spherical organisms, some of which are larger and have internal structures. In the bottom left, there is another cluster of spherical organisms. In the bottom right, there is a spherical organism with radiating lines. In the center, there are two elongated, rod-shaped organisms with small protrusions at their ends. The text "Experimental Design" is centered in the middle of the image in a white, bold, sans-serif font.

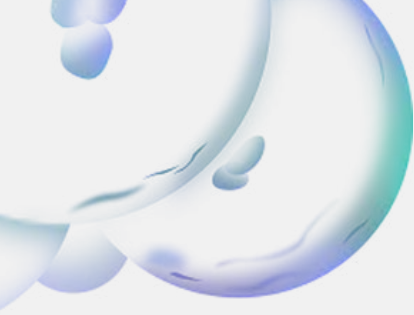
Experimental Design



Hypothesis

“Different protein sequence encoding methods will have varying impacts on the performance of deep learning models in predicting EC numbers”



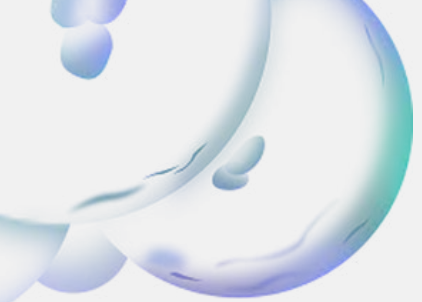


Experimental Design

EC Number

- We limited our classification to EC number class 1 at the second level to focus the project within one month.
- Consistent preprocessing and input formatting were applied to ensure fair comparisons across all experiments.





Experimental Design

EC Number

- We limited our classification to EC number class 1 at the second level to focus the project within one month.
- Consistent preprocessing and input formatting were applied to ensure fair comparisons across all experiments.

Amino Acid Sequence

- All protein sequences were set to a fixed length of 512.
 - Sequences shorter than 512 were padded with special tokens.
 - Vocabulary consisted of 20 amino acids plus 2 special tokens, totaling 22 unique tokens.
- 

Experimental Design 1: One-Hot

One-hot

ESPIC-based code for preprocessing

'A' $\rightarrow$ [0, 1, 0, ..., 0]

We expect strong performance from **CNNs** on one-hot encoded sequence data.

1. Define Amino Acid Categories

'A', 'C', ..., 'Y', 'X'



2. Sequence Encoding



3. Padding or Truncation

Sequence Max-Length : 512



4. Character-wise One-hot Encoding



Experimental Design 2

: K-mer & Tokenization

Amino acid sequence: MAFSAEDVLKEY

K-mer

Adapted from the DNABERT paper

[MAF , AFS , FSA , SAE]

It groups **3 consecutive amino acids into a single token**, allowing the model to capture local sequential patterns akin to "words" in natural language.

Tokenization

A simplified approach compared to k-mer

[M , A , F , S , A , E , D , V , L , K , E , Y]

It treats each amino acid as an **independent unit**, effectively considering them in isolation.



K-mer

Sequence Sliding Window

[MAF , SAE , DVL , KEY]



Tokenization

Amino Acid Parsing

[M , A , F , S , A , E , D , V , L , K , E , Y]



Vocabulary Mapping

: Unique integer IDs were assigned to each token observed in the data.



Sequence Indexing

: Tokenized sequences were converted into sequences of these integer IDs.



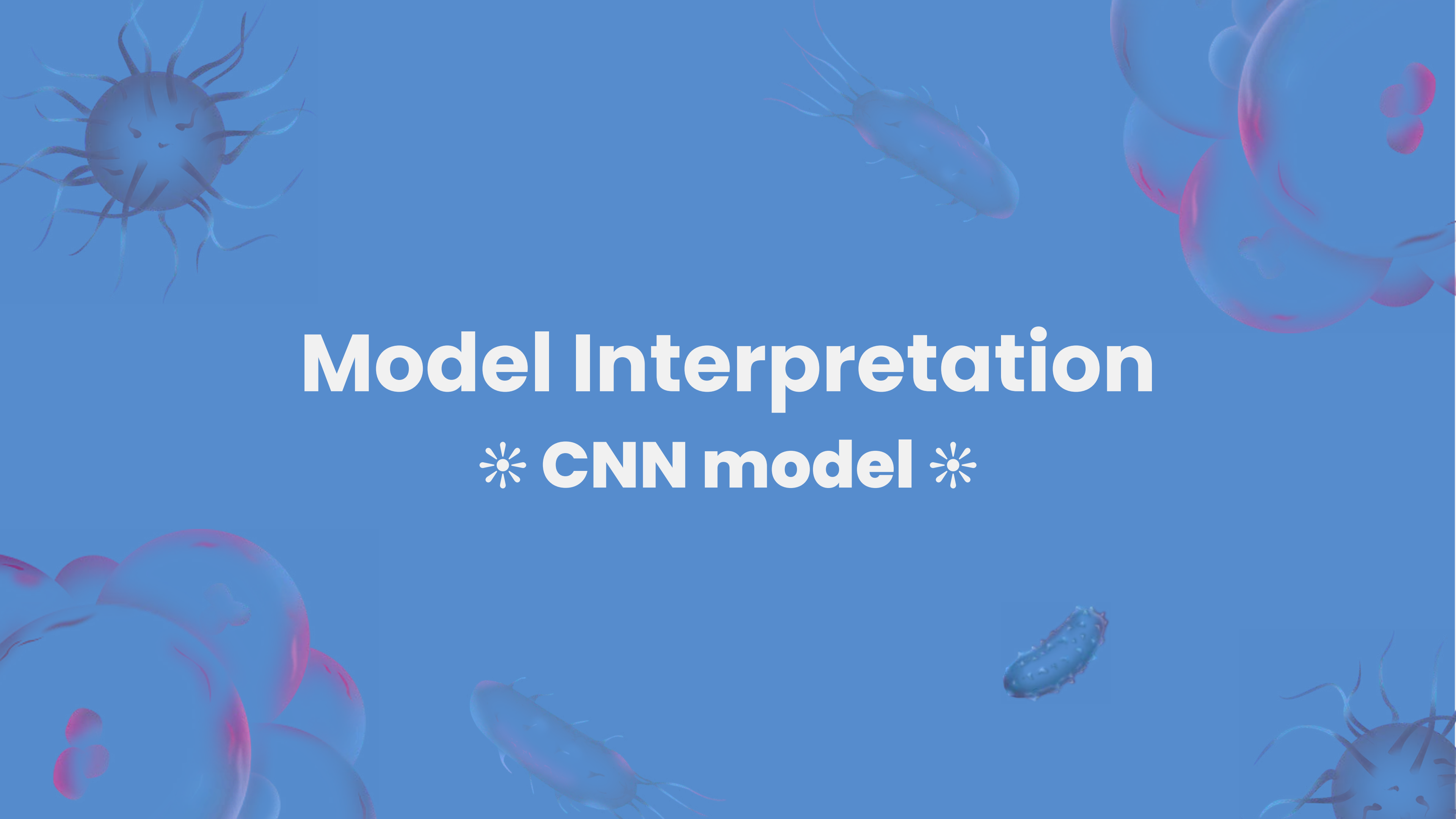
Padding and Truncation

: Sequences were padded or truncated to a fixed length(152) for uniform input size.



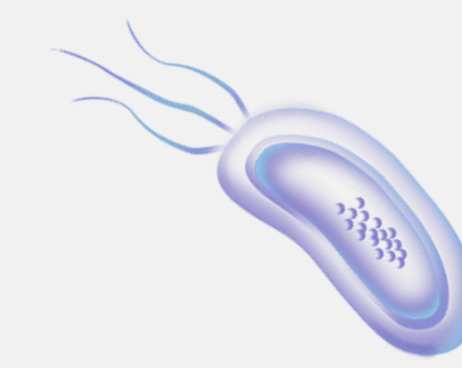
Embedding Input

: Indexed sequences were mapped to dense vectors via an embedding layer for model input.

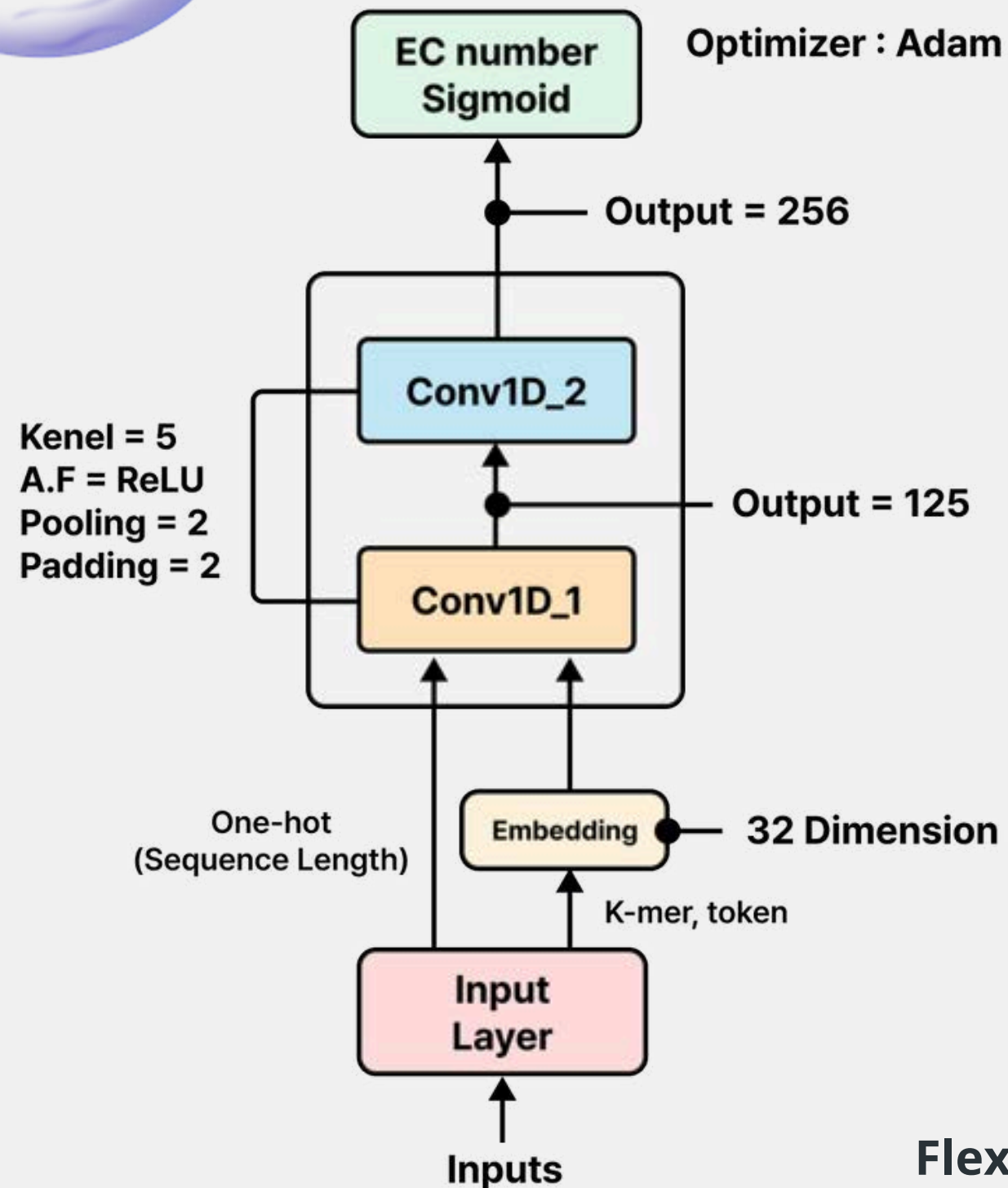


Model Interpretation

✱ **CNN model** ✱



CNN Model



CNN-based protein-function predictor can be framed as a four-stage pipeline:

1. Input transformation by mode



2. Two-stage convolution

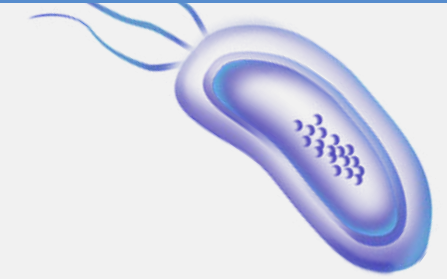


3. Flatten to 1D vector



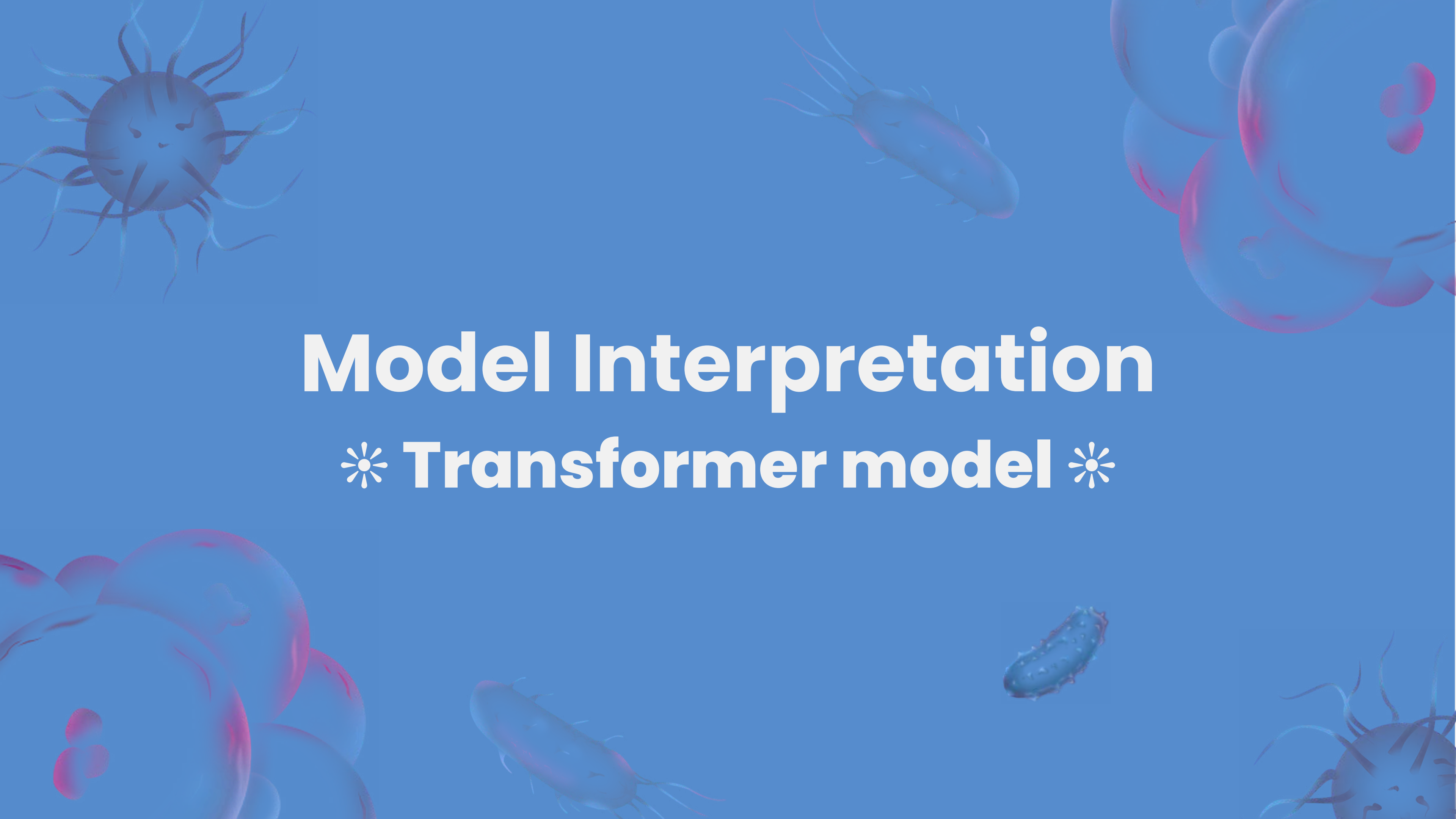
4. Predict class scores

All input types processed with **1D CNN** for a unified, consistent model
Flexible pattern extraction from different encodings within a single framework



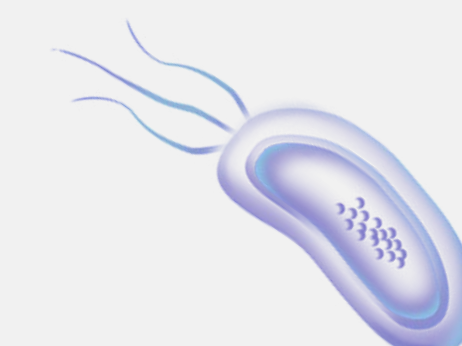
Experimental Results

	CNN Macro F1-Score	CNN Micro F1-Score
One-hot	0.8387	0.8799
K-mer	0.7491	0.8199
Tokenization	0.8274	0.8754

The background is a solid blue color with several stylized, semi-transparent illustrations of microorganisms. In the top left, there is a large, dark blue spherical virus with many thin, radiating spikes. In the top right, there are several red, oval-shaped bacteria with small flagella. In the bottom left, there are more red, oval-shaped bacteria. In the bottom right, there is a large, dark blue spherical virus with many thin, radiating spikes. In the center, there are several red, oval-shaped bacteria with small flagella. In the bottom center, there is a small, dark blue, rod-shaped bacterium with small flagella.

Model Interpretation

✱ **Transformer model** ✱



Transformer Model

Transformer-based protein-function predictor can be framed as a three-stage pipeline:

1. Residue-level contextual embedding extraction

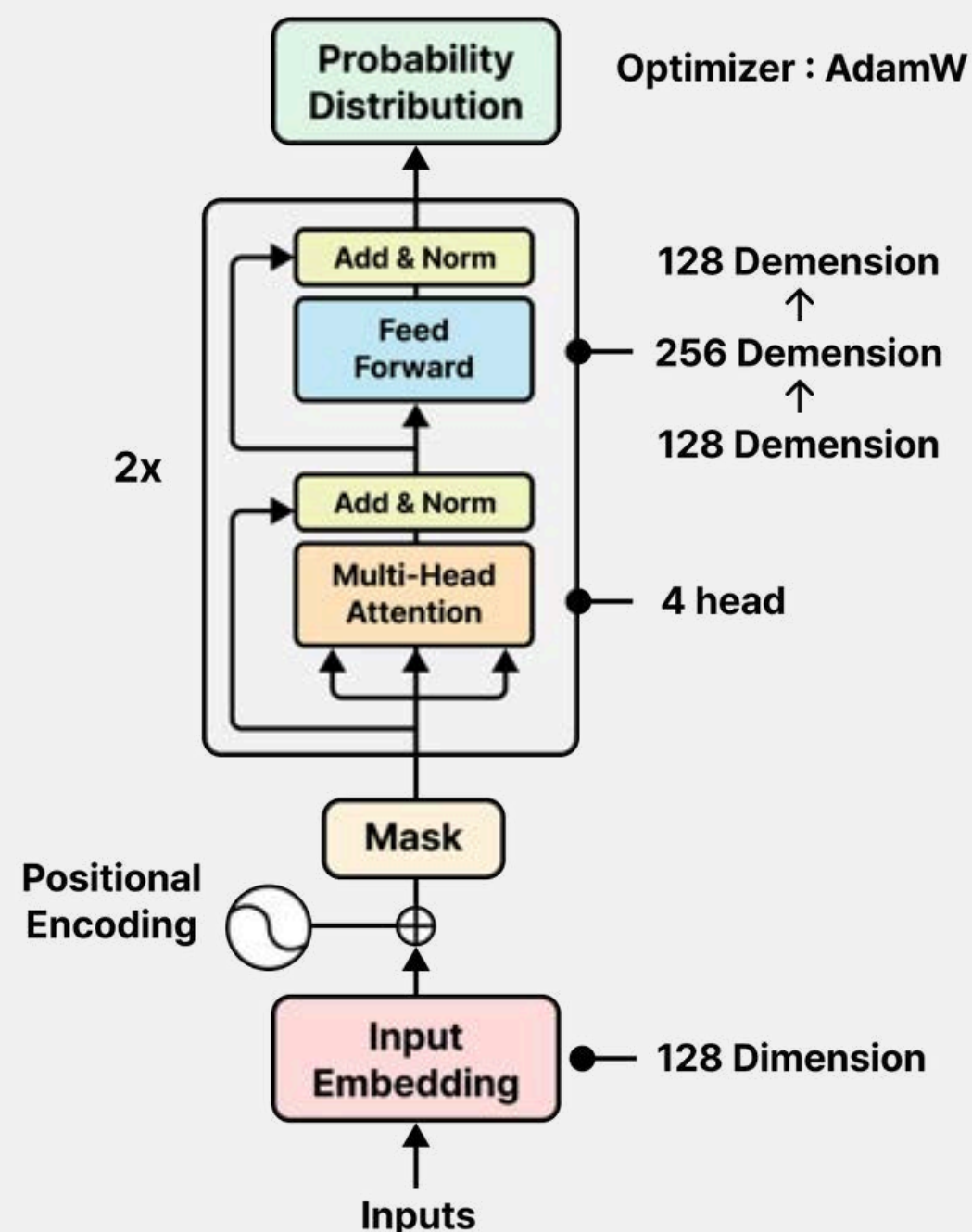


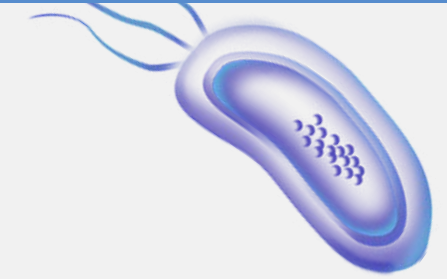
2. Global sequence representation learning via self-attention



3. Supervised fine-tuning / classification

Transformer models encode functional signatures by capturing **long-range dependencies** and **global evolutionary** context across the **entire sequence**.





Experimental Results

	Transformer Macro F1-Score	Transformer Micro F1-Score
One-hot	0.6279	0.7191
K-mer	0.8030	0.8761
Tokenization	0.4527	0.5099

The background is a solid teal color with several 3D-rendered microscopic organisms. In the top left, there is a large, spherical, pinkish-purple cell with many thin, radiating filaments. In the top right, there are several large, translucent, light blue spherical cells, some of which contain smaller, darker blue structures. In the bottom left, there is a cluster of similar light blue spherical cells. In the bottom right, there is a large, spherical, pinkish-purple cell with radiating filaments, similar to the one in the top left. Scattered throughout the background are several rod-shaped bacteria, some in light blue and some in a darker blue/purple color, all with fine, hair-like appendages (flagella) extending from their surfaces.

Experimental Results

Macro & Micro F1-Score Analysis



Macro F1-Score

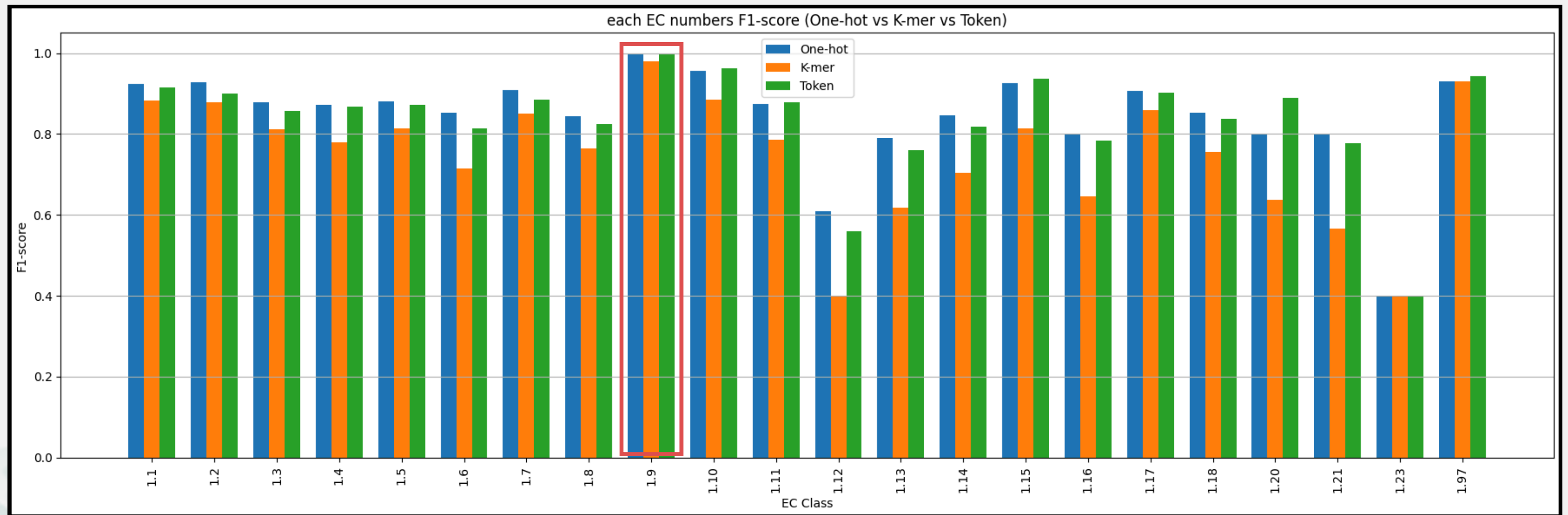
: measures global performance by computing metrics over all instances, treating each prediction equally regardless of class.

Micro F1-Score

: computes the F1-score for each class independently and averages them, giving equal weight to all classes.

	CNN Macro F1-Score	CNN Micro F1-Score	Transformer Macro F1-Score	Transformer Micro F1-Score
One-hot	0.8387	0.8799	0.6279	0.7191
K-mer	0.7491	0.8199	0.8030	0.8761
Tokenization	0.8274	0.8754	0.4527	0.5099

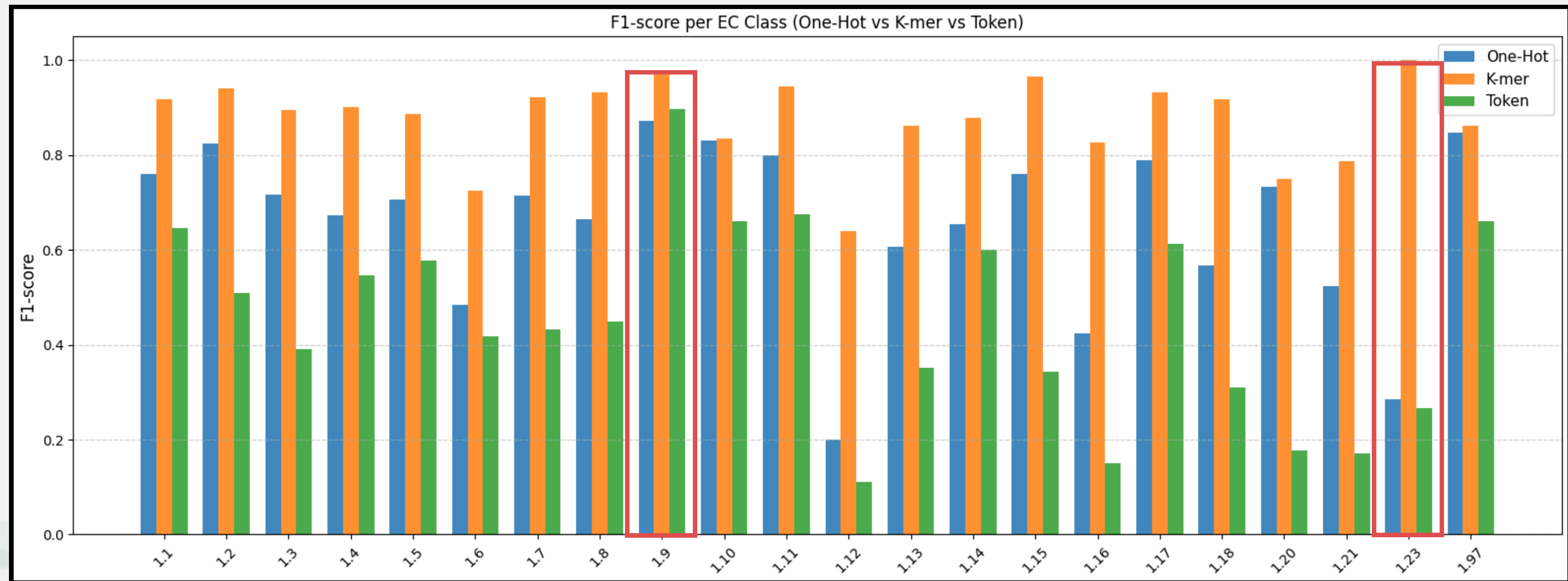
CNN Performance Analysis by EC Number



CNN Performance Analysis by EC Number

1. Most classes achieve high F1-scores (0.8–1.0), indicating **strong overall CNN performance**.
2. **Class-wise performance varies**
 - a. Class 1.9 shows near-perfect scores across all encodings.
 - b. Classes 1.12, 1.13, 1.16, 1.20, and 1.21 show lower F1-scores, suggesting classification difficulty.
3. Encoding-wise performance:
 - a. **Tokenization outperforms** others in several classes.
 - b. **One-hot encoding is consistently stable**.
 - c. **K-mer encoding underperforms** in **certain cases**.

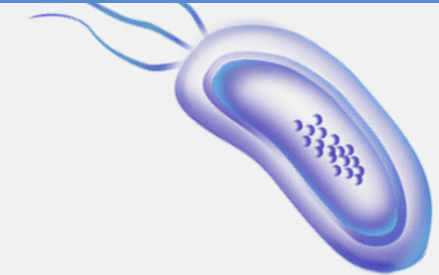
Transformer Performance Analysis by EC Number



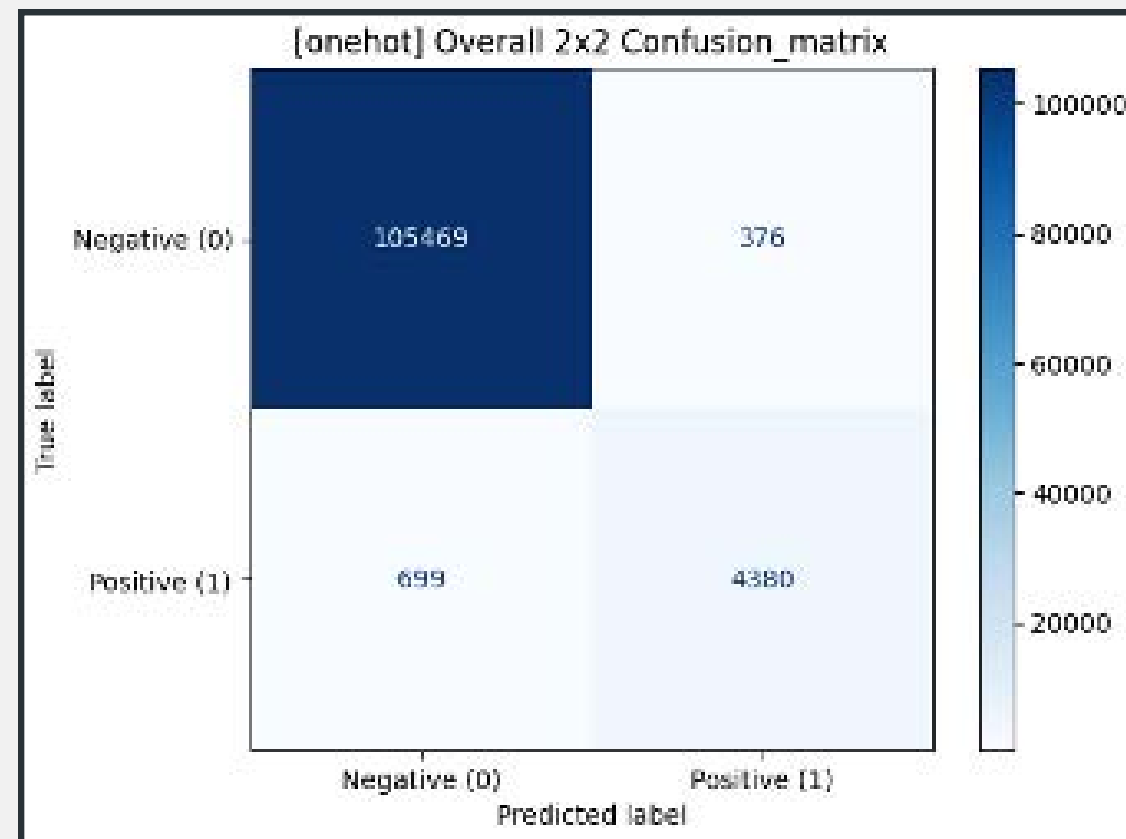
Transformer Performance Analysis by EC Number

1. Most EC classes show the highest F1-scores with **K-mer encoding**, suggesting it is the **most effective representation in this setup**.
2. Class-wise insights:
 - a. Class 1.9 and 1.23 reach near-perfect scores with K-mer.
 - b. Classes like 1.11, 1.12, 1.13, 1.16, 1.20, and 1.21 show relatively low scores across all encodings, indicating these are harder to classify.
3. Encoding-wise performance:
 - a. **K-mer consistently outperforms** One-hot and Tokenization in most classes.
 - b. **Tokenization shows weaker performance**, especially in classes like 1.12, 1.13, and 1.16
 - c. **One-hot performs moderately** but lags behind K-mer in general.

CNN Confusion Matrix

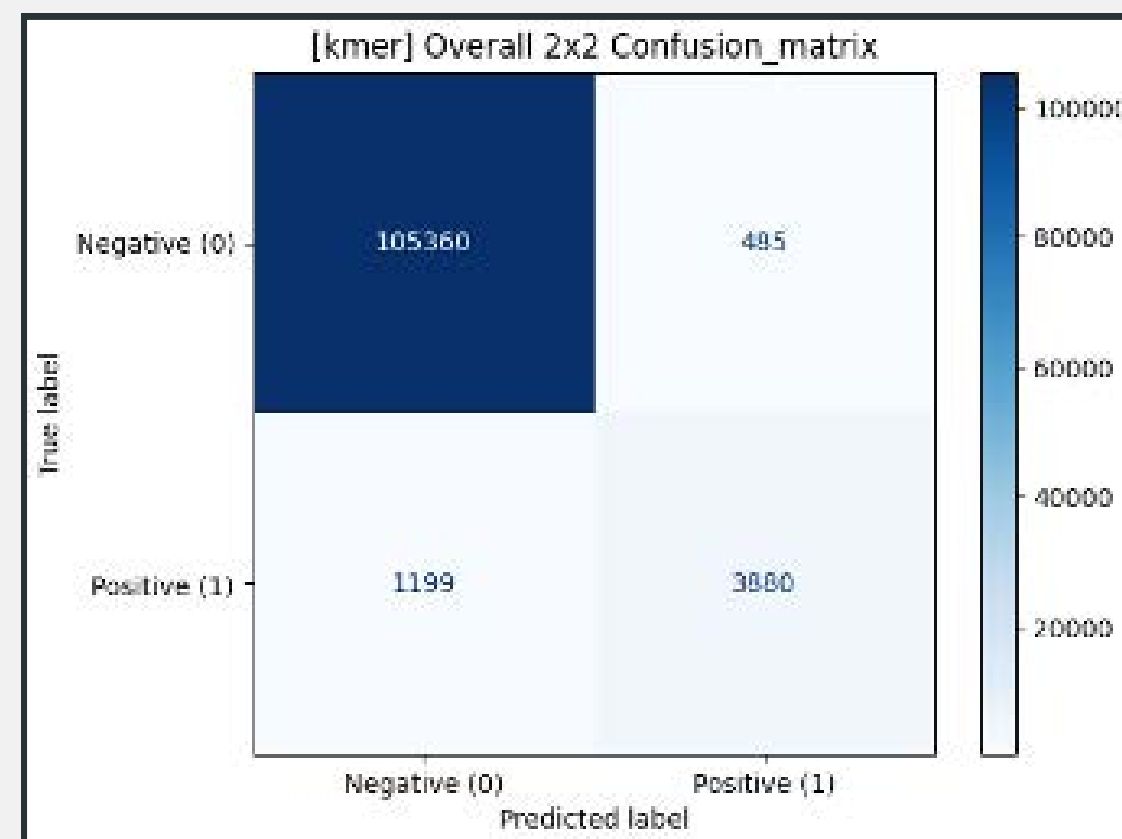


One-hot



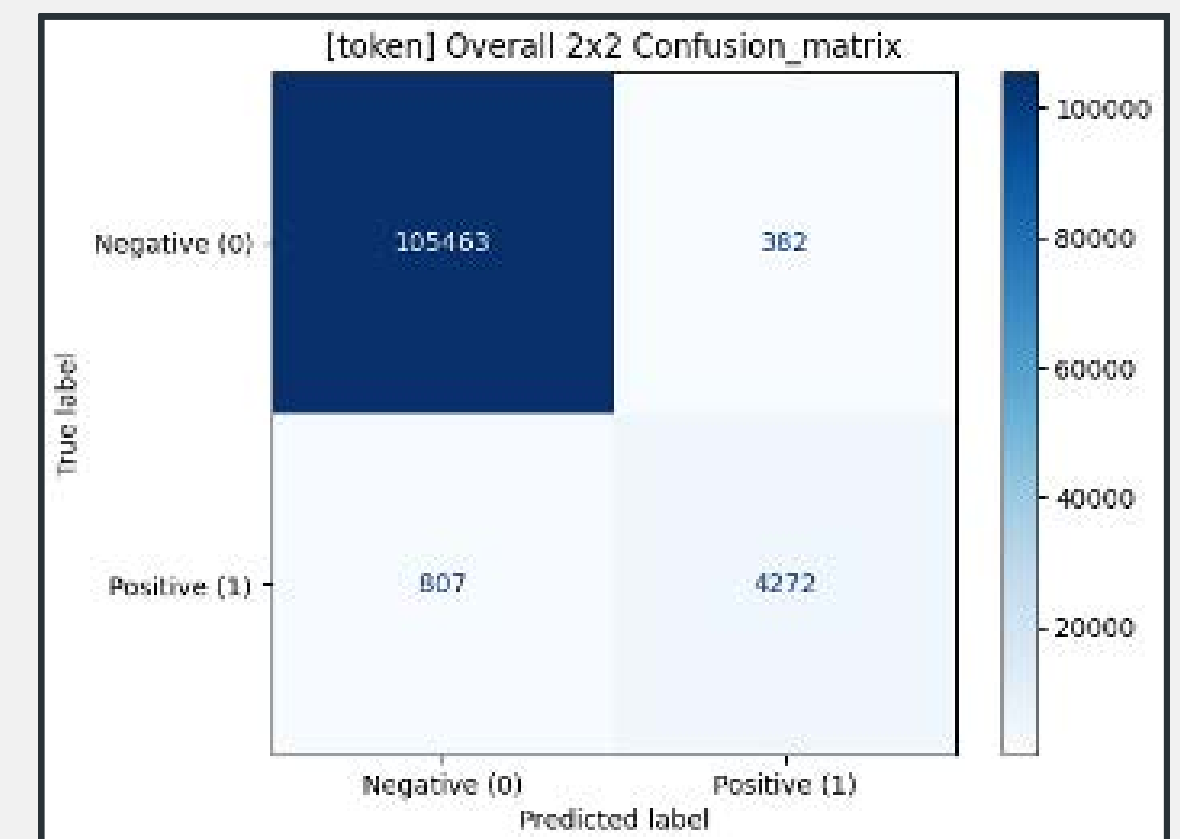
Precision $\approx$ 92.10%
Recall $\approx$ 86.23%
F1 $\approx$ 89.00%
Accuracy $\approx$ 99.03%

K-mer

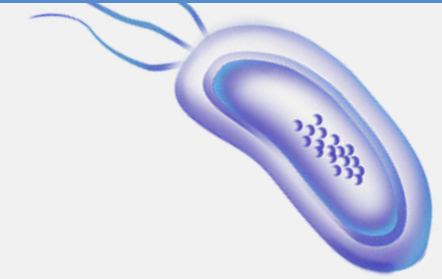


Precision $\approx$ 88.89%
Recall $\approx$ 76.39%
F1 $\approx$ 82.20%
Accuracy $\approx$ 98.48%

Tokenization



Precision $\approx$ 91.79%
Recall $\approx$ 84.08%
F1 $\approx$ 87.87%
Accuracy $\approx$ 98.93%



CNN Confusion Matrix

One-hot

Precision $\approx$ 92.10%

Recall $\approx$ 86.23%

F1 $\approx$ 89.00%

Accuracy $\approx$ 99.03%

K-mer

Precision $\approx$ 88.89%

Recall $\approx$ 76.39%

F1 $\approx$ 82.20%

Accuracy $\approx$ 98.48%

Tokenization

Precision $\approx$ 91.79%

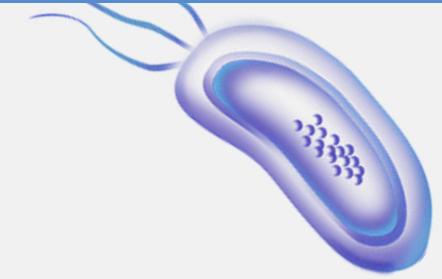
Recall $\approx$ 84.08%

F1 $\approx$ 87.87%

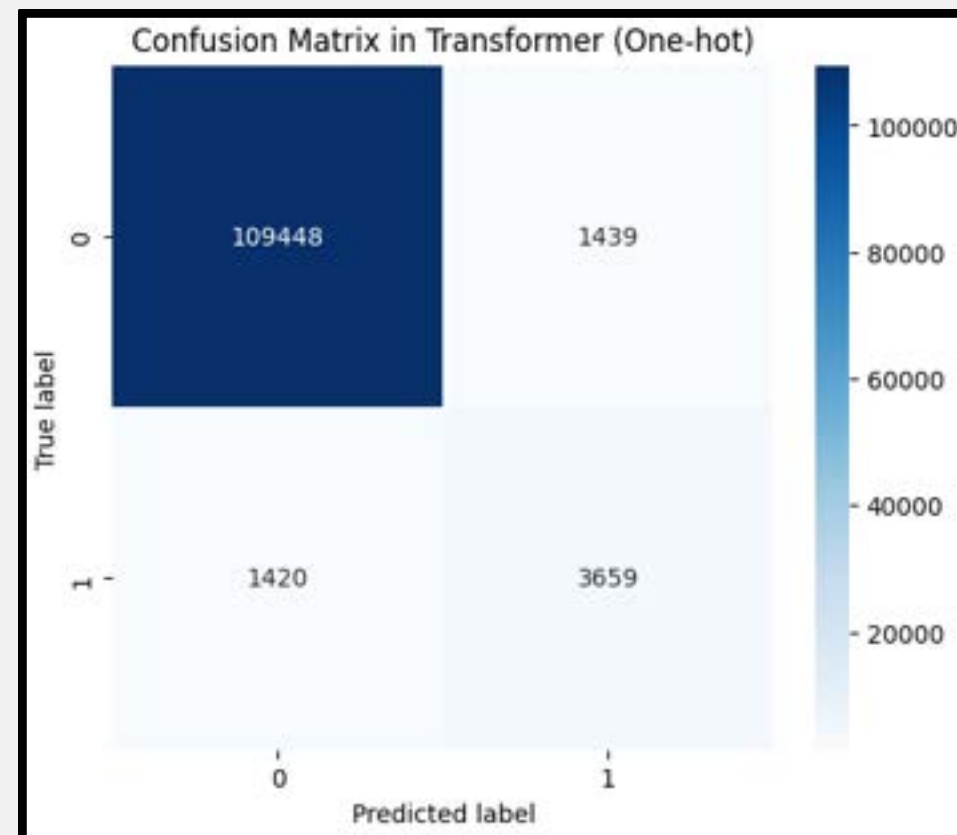
Accuracy $\approx$ 98.93%

- **One-hot encoding performs best overall**, achieving the highest F1-score and highest accuracy.
- **Tokenization follows closely**, with strong precision and decent recall.
- **K-mer** encoding shows the **weakest performance**, particularly in recall, indicating it missed more positive cases.
- Trade-off between Precision and Recall:
 - One-hot and Tokenization have a better balance.
 - K-mer has lower recall $\rightarrow$ more false negatives.

Transformer Confusion Matrix

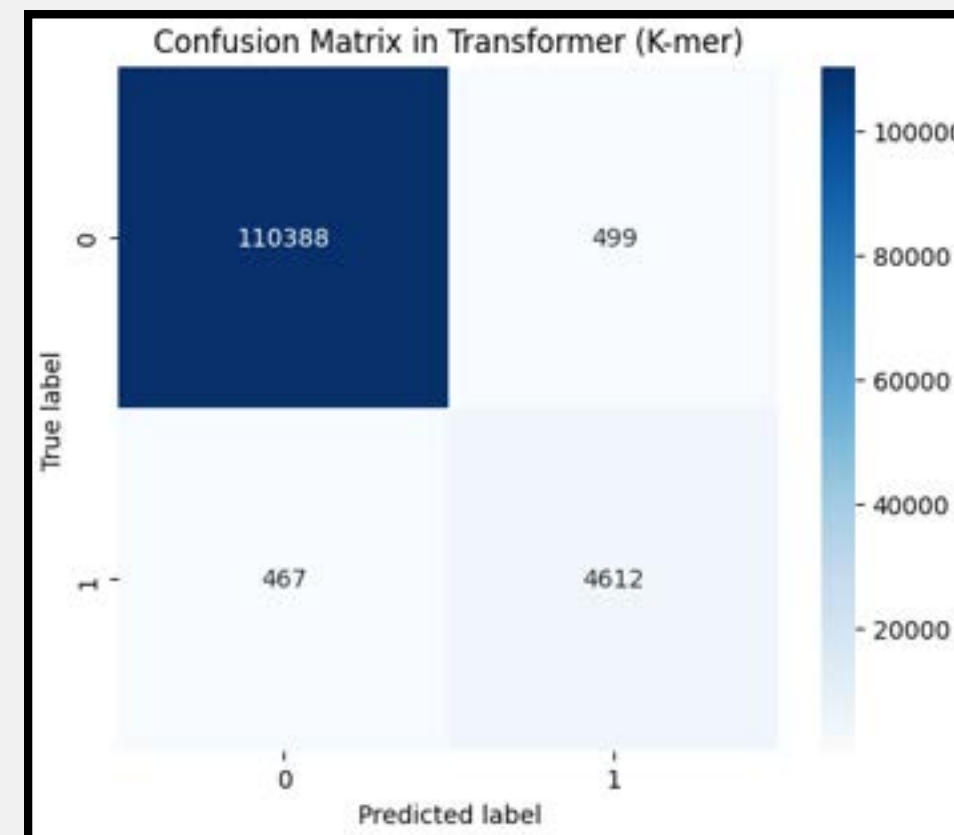


One-hot



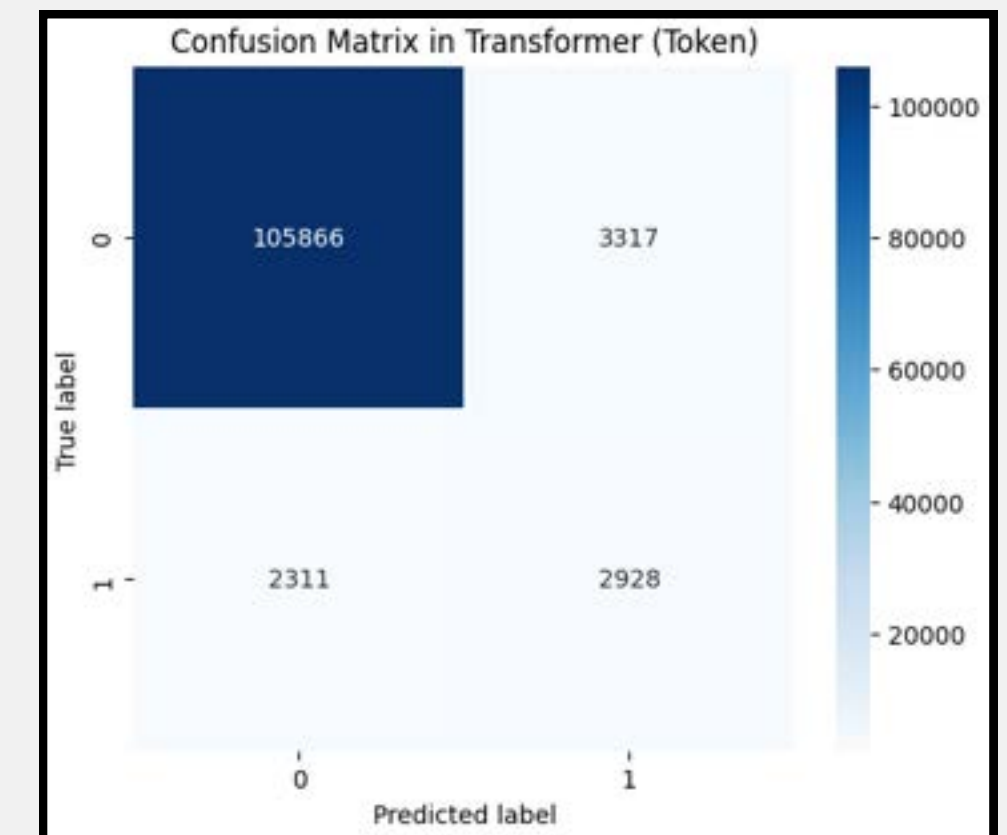
Precision $\approx 71.79\%$
Recall $\approx 72.06\%$
F1 $\approx 71.92\%$
Accuracy $\approx 96.70\%$

K-mer



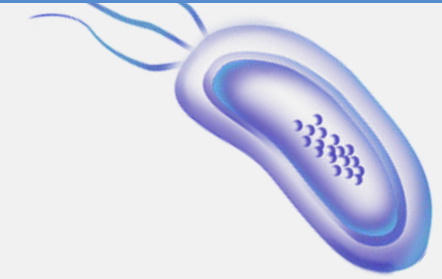
Precision $\approx 90.23\%$
Recall $\approx 90.79\%$
F1 $\approx 90.49\%$
Accuracy $\approx 99.21\%$

Tokenization



Precision $\approx 46.87\%$
Recall $\approx 55.89\%$
F1 $\approx 50.97\%$
Accuracy $\approx 95.08\%$

Transformer Confusion Matrix



One-hot

Precision $\approx 71.79\%$

Recall $\approx 72.06\%$

F1 $\approx 71.92\%$

Accuracy $\approx 96.70\%$

K-mer

Precision $\approx 90.23\%$

Recall $\approx 90.79\%$

F1 $\approx 90.49\%$

Accuracy $\approx 99.21\%$

Tokenization

Precision $\approx 46.87\%$

Recall $\approx 55.89\%$

F1 $\approx 50.97\%$

Accuracy $\approx 95.08\%$

- **one-hot encoding yields moderate performance**, shows a fair balance between precision and recall, but lacks the ability to capture rich sequence context
- **K-mer encoding achieves the best performance**, effectively capturing sequence patterns for accurate classification.
- **Tokenization performs poorly**, with low F1-score of and the lowest precision and recall among the three. Although the overall accuracy appears high , it is likely inflated by class imbalance.

The Impact of K-mer-Based Input on Deep Learning Models



1. Capturing Local Sequence Patterns

: K-mer (k=3) splits amino acid sequences into overlapping trigrams, effectively capturing local patterns. These subsequences often align with functional motifs or catalytic sites.

- CNNs are well-suited for detecting such local features, making k-mer input highly compatible with convolutional filter
- Transformers benefit from meaningful input units — k-mers increase information density while retaining local context through positional encoding and self-attention.

Amino acid sequence: MAFSAEDVLKEY



[MAF , AFS , FSA , SAE]

The Impact of K-mer-Based Input on Deep Learning Models



2. Information Compression and Redundancy Reduction

: Tokenizing sequences by single amino acids ($k=1$) leads to low information density and longer input lengths, increasing computational cost, especially in Transformers with quadratic complexity.

- K-mer ($k=3$) reduces input length by two-thirds and provides more informative learning units, improving efficiency and performance.

< Tokenization >

[M , A , F , S , A , E , D , V , L , K , E , Y]

< K-mer >

[MAF , AFS , FSA , SAE]

Incorporating EC Number Similarity into Loss Design



By designing a loss function that incorporates the similarity between the predicted and true EC numbers,

predictions that are functionally similar to the correct EC number can be **penalized less**, while predictions from entirely **different classes** can be **penalized more heavily**.

This encourages the model's output to be **aligned with functional similarity**, ultimately guiding the learning process in a biologically meaningful direction.

Discussion

Discussion

1. Despite GPU acceleration, the computation cost remains high.
2. EC-level performance varies with sequence length and data availability.
3. Both CNN and Transformer models underperform on certain EC classes, suggesting the need to explore K-fold cross-validation for potential improvement.

Reference

- Lord, P. W., Stevens, R. D., Brass, A., & Goble, C. A. (2003). Investigating semantic similarity measures across the Gene Ontology: the relationship between sequence and annotation. *Bioinformatics* <https://pmc.ncbi.nlm.nih.gov/articles/PMC2709532/>
- datax-lab. (2023). ECPICK: Biologically interpretable deep learning enhances trustworthy enzyme commission number prediction and discovers potential motif sites [Source code]. GitHub. <https://github.com/datax-lab/ECPICK>
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *International Conference on Learning Representations*. <https://arxiv.org/abs/2010.11929>
- Yanrong Ji, Zhihan Zhou, Han Liu, Ramana V Davuluri, (2021) DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome <https://doi.org/10.1093/bioinformatics/btab083>
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin. Attention Is All You Need (2017) <https://arxiv.org/abs/1706.03762>

Individual Contribution

Jueun Park: Tokenization preprocessing, transformer model training(K-mer), confusion matrix, K-mer impact analysis

Minjae Lee: kmer preprocessing, transformer model training(k-mer)

Jihee Kang: kmer preprocessing, transformer model training(one-hot)

Soyoung Kim: Analyzed the CNN architecture from the EC PICK paper and devised a one-hot encoding strategy tailored to the project's objective

Pyeonghwa Jang: One-hot preprocessing, CNN model design and training(One-hot, K-mer, Tokenization)

**Thankyou
for listening**